

Homework 1

Due by: 1.6.2026, 23:59

General Guidelines

1. The main goal of this homework is to experiment with C++ concurrency and smart pointers.
2. Don't use any external libraries for this homework, except native C++ libs.
3. You should use only C++11 for exercises 1-3. You may use C++14 for exercise 4.
4. Make sure to create new classes and files with the exact names defined ahead. Otherwise, you will face compilation errors during automatic testing.
5. The homework assignment is 100% auto graded. You may resubmit your solution to gradescope multiple times before the due date of this homework assignment.
6. This homework assignment includes several .h files. Read ahead.
7. In total, you should upload (together) these files to gradescope:
 - a. BoundedQueue_impl.h
 - b. BoundedQueue1p1c_impl.h
 - c. UnboundedQueue1p1c_impl.h
 - d. ConnectionPool_Impl.h

Exercise 1: parallel bounded queue – 20 points

Implement a parallel bounded FIFO queue. All its memory should be allocated during queue creation.

1. The queue capacity should NOT grow at any point in time.
2. Implement a “*BoundedQueue*” C++ class which implements the bounded queue. It should inherit the abstract class in the *BoundedQueue.h* file (you are still allowed to extend your class with additional private member variables and functions).
3. Notice the semantics of pop and push, as defined in *BoundedQueue.h*.
4. Take good care to implement correct synchronization.

Upload *BoundedQueue_impl.h* file with your class implementation to gradescope.

Exercise 2: parallel bounded queue with one producer /one consumer – 20 points

Implement another version of a parallel bounded FIFO queue which is known to be used by at most one producer (push) and one consumer (pop)

1. All its memory should be allocated during queue creation.
2. The queue capacity should NOT grow at any point in time.
3. Implement a “*BoundedQueue1p1c*” C++ class for this bounded queue. It should inherit the abstract class in the *BoundedQueue1p1c.h* file (you are still allowed to extend your class with additional private member variables and functions).
4. Notice the difference in behavior of *push* and *pop* compared with Exercise 1
 - With *pop*, if the queue is considered empty, return false. Otherwise, return true.
 - With *push*, if the queue is considered full, return false. Otherwise, return true.
5. Think hard how to leverage the assumption of one customer/producer to minimize synchronization.

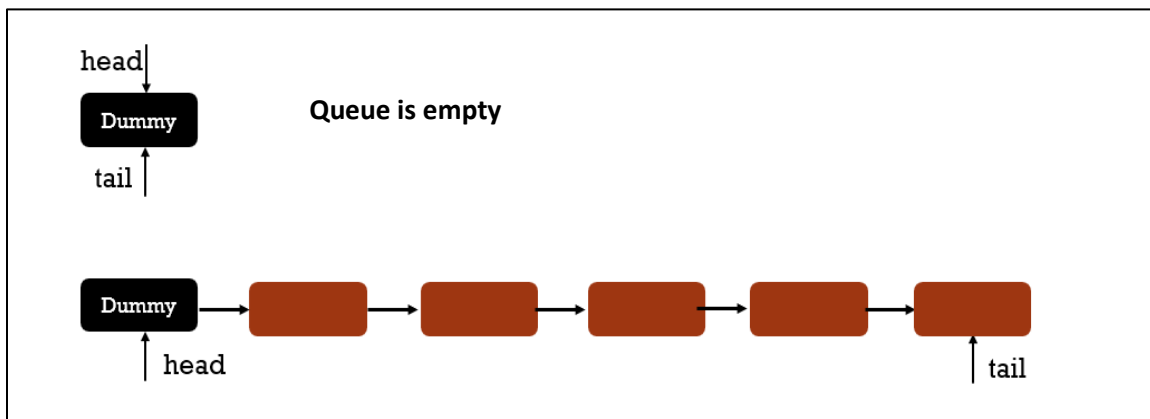
- You are NOT allowed to use any mechanism of blocked synchronization (e.g., mutexes) in your solution, whether directly (e.g., using C++ mutexes) or indirectly (i.e., implementing blocking locks by yourself)

Upload *BoundedQueue1p1c_impl.h* file with your class implementation to gradescope.

Exercise 3: parallel unbounded queue with one producer /one consumer – 30 points

Implement a parallel unbounded FIFO queue with at most one producer and one customer.

- Implement an “*UnboundedQueue1p1c*” C++ class for the unbounded queue. It should inherit the abstract class in the *UnboundedQueue1p1c.h* file (you are still allowed to extend your class with additional member variables and functions).
- Notice the difference in behavior of *push* and *pop* compared with the other exercises:
 - With *pop*, if the queue is empty, return false. Otherwise, return true.
 - Push always succeeds.
- Your implementation should be based on link lists in C (not C++). Any use of C++ STL containers or arrays is NOT allowed.
- Think hard how to leverage the assumption of one customer/producer to minimize synchronization.
 - You are NOT allowed to use any mechanism of blocked synchronization, exactly as in exercise #2.
 - Hint: use a dummy node (see following figure)
 - Upon queue initialization, it already contains one (dummy) element (no value inside). Moreover, during the queue lifetime, it may happen that other nodes will become dummy.
 - The queue never gets empty. Pop fails if the queue has only one element (dummy)
 - Tail points to latest element in the queue. Push is always done immediately after the tail element.
 - Pop is always done on head->next (NOT on head).
 - Convince yourself that the use of dummy elements helps you to reduce synchronization for the case of an empty queue



Upload *UnboundedQueue1p1c_impl.h* file with your class implementation to gradescope.

Exercise 4: Smart pointers - 30 points

Imagine you're designing a simple connection pool for a remote database system. A connection is an object by which a client code can access a remote database. Connections are expensive to create, so the pool maintains a limited number of reusable connection objects. Your task is to use smart pointers (`std::unique_ptr` and `std::shared_ptr`) to manage these connection objects efficiently.

Individual connections are of type *Connection* (defined in *Connection.h*). The connection pool is defined by a *ConnectionPool* class which should be inherited from the *ConnectionPoolAbstract* class (see *Connection.h*).

The *ConnectionPool* class should have only the following public methods:

1. Constructor – initialize the pool with connections. The pool size equals the *poolSize* parameter of the constructor.
2. `borrowConnection` – allows a thread to get a connection. This connection can later be shared by other threads. Once all threads complete their execution, this connection is automatically returned to the connection pool. If the pool is empty (i.e., all connections are borrowed), the requester thread should wait until one of the connections becomes available.
3. TIP: define a custom deleter function for the shared pointers to a connection borrowed from the pool. This custom deleter should automatically return that connection back to the pool once the last shared pointer to that connection is destroyed. See https://en.cppreference.com/w/cpp/memory/shared_ptr/shared_ptr (or learn and explore custom deleters via AI tools).
4. **You may use C++14 for this exercise only.**

Upload *ConnectionPool_Impl.h* file with your class implementation to gradescope.

Good luck!